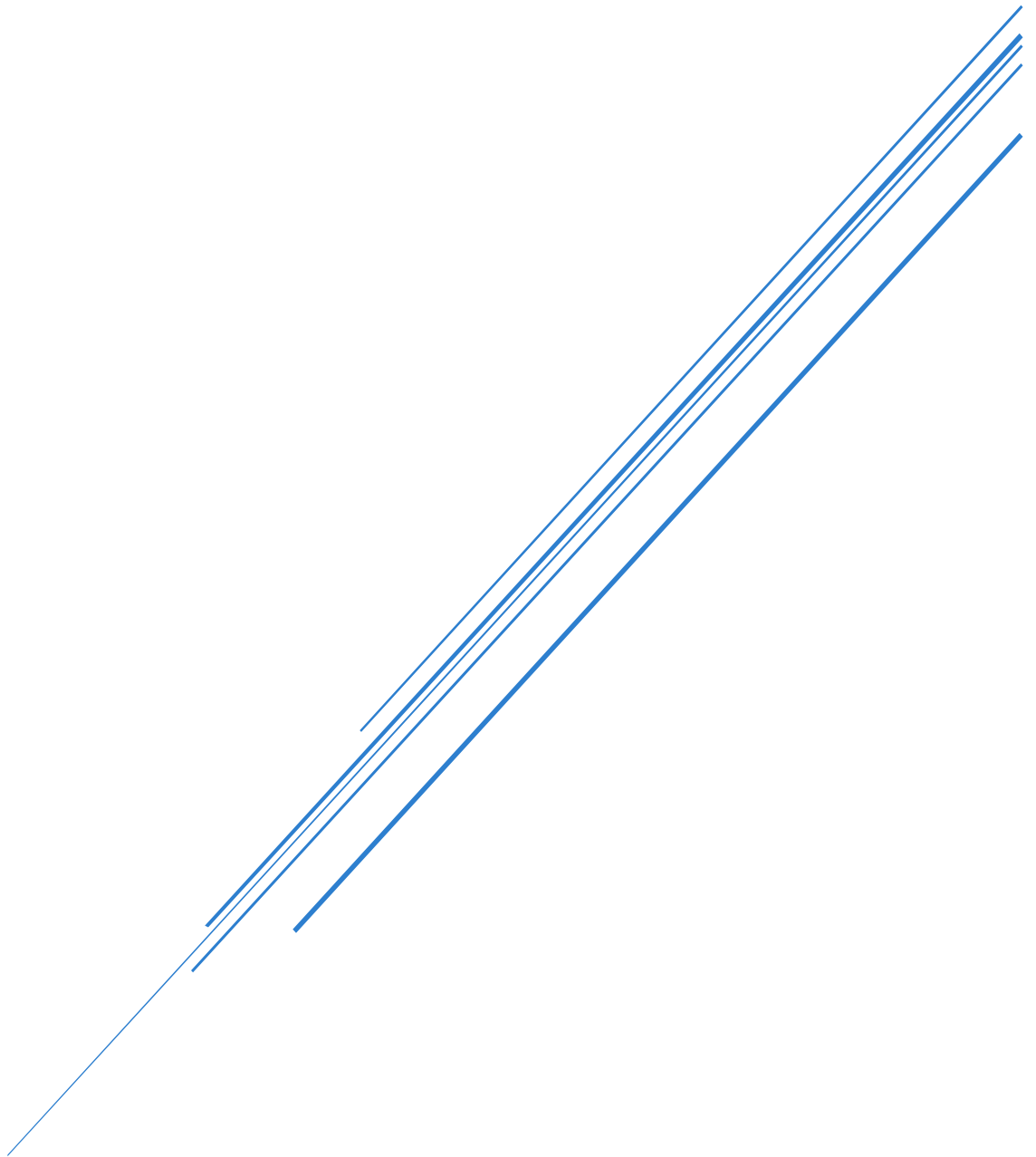


INFORME RENDIMIENTO

SP5



Grupo A

Índice

CSV vs Parquet	2
1. CSV (Original) — 5.20 s 1.52 MB	2
2. Parquet sin compresión — 0.76 s 527.30 KB	2
3. Snappy (comprimido) — 0.44 s 527.47 KB	2
4. Predicate Pushdown con baja cardinalidad (gender) — 0.43 s	3
5. Predicate Pushdown con alta cardinalidad (student_id) — 0.55 s	3
6. Particionado por género — 0.37 s 617.67 KB	3
7. Schema Merging — 0.57 s	4
8. Bucketing (JOIN) — 1.17 s	4
Resultados finales	4

CSV vs Parquet

Dataset: student_productivity_distraction_dataset_20000.csv

Registros: 20.000

Columna de particionado: gender

Consulta de prueba: filtrar por gender == "Female" y sumar productivity_score

1. CSV (Original) — 5.20 s | 1.52 MB

El CSV es un formato de texto plano que no está pensado para consultas rápidas. Cuando Spark lo lee, tiene que ir línea a línea parseando el texto y convirtiendo cada campo a su tipo correspondiente (enteros, floats, strings...). Lo peor es que, aunque nosotros solo necesitemos dos columnas (gender y productivity_score), Spark se ve obligado a cargar la fila entera con las 18 columnas, porque el CSV no tiene forma de "saltar" a una columna concreta. Es como tener que leer un libro entero para buscar una sola palabra en cada página. De ahí que sea con diferencia la prueba más lenta.

2. Parquet sin compresión — 0.76 s | 527.30 KB

Aquí es donde se nota el cambio a almacenamiento columnar. Parquet guarda los datos por columnas en vez de por filas, así que cuando le pedimos solo gender y productivity_score, Spark lee únicamente esas dos columnas del disco e ignora las otras 16. Solo con este cambio ya nos ahorramos un 66% de espacio en disco y el tiempo baja de 5.20 s a 0.76 s, que es casi 7 veces más rápido. Además, al estar los valores de cada columna juntos y ser del mismo tipo, la compresión interna del formato funciona mucho mejor que en un CSV donde cada fila mezcla strings con números.

3. Snappy (comprimido) — 0.44 s | 527.47 KB

Snappy es un algoritmo de compresión que se usa mucho en Big Data porque es muy rápido descomprimiendo. El tamaño en disco es prácticamente igual al Parquet sin comprimir (en nuestro caso la diferencia es mínima porque el dataset ya era pequeño), pero el tiempo baja a 0.44 s. ¿Por qué mejora la velocidad si tiene que descomprimir? Porque al ocupar menos en disco, los datos viajan más rápido del disco a la RAM, y el cuello de botella en muchas consultas no es la CPU sino el I/O de disco. Es el equilibrio que busca Snappy: comprimir lo justo para que lo que ganamos en lectura compense lo que gastamos en descompresión.

4. Predicate Pushdown con baja cardinalidad (gender) — 0.43 s

Aquí activamos el Predicate Pushdown, que es una optimización donde Spark "empuja" el filtro hasta la capa de almacenamiento. Parquet guarda metadatos en cada bloque (row group) indicando qué valores contiene. Cuando filtramos por `gender == "Female"`, Spark mira esos metadatos y si un bloque solo tiene hombres, directamente lo salta sin leerlo. El problema con `gender` es que tiene baja cardinalidad (solo 3 valores), así que la mayoría de bloques van a contener mezcla de los tres géneros y el pushdown no puede descartar muchos. Por eso la mejora respecto a Snappy es pequeña, de 0.44 s a 0.43 s.

5. Predicate Pushdown con alta cardinalidad (student_id) — 0.55

s

Con `student_id` buscamos un registro concreto (el estudiante 5000). Como los IDs tienen alta cardinalidad (20.000 valores distintos), Parquet puede usar las estadísticas min/max de cada bloque de forma mucho más efectiva. Si un bloque tiene IDs del 1 al 5000 y otro del 5001 al 10000, Spark sabe exactamente en cuál buscar. El tiempo es un poco mayor que en el caso anterior (0.55 s vs 0.43 s), pero hay que tener en cuenta que la consulta es distinta: aquí estamos buscando una aguja en un pajar (1 registro de 20.000), mientras que antes filtrábamos por un grupo que representa un tercio del dataset.

6. Particionado por género — 0.37 s | 617.67 KB

Esta es la optimización más potente para nuestro caso. Al escribir los datos con `partitionBy("gender")`, Spark crea carpetas separadas en disco: `gender=Female/`, `gender=Male/` y `gender=Other/`. Cuando después filtramos por `gender == "Female"`, Spark ni siquiera abre las carpetas de `Male` y `Other`. Es lo que se llama Partition Pruning: en vez de leer un archivo y buscar dentro, directamente va a la carpeta correcta. El tiempo baja a 0.37 s, que es 14 veces más rápido que el CSV original. El tamaño sube un poco respecto al Parquet sin particionar (617 KB vs 527 KB) porque al dividir en carpetas se añaden metadatos y el archivo de cada partición tiene su propia cabecera, pero merece la pena por la ganancia en velocidad.

Elegimos `gender` como columna de particionado porque tiene solo 3 valores distintos. Si hubiésemos particionado por `student_id` (20.000 valores), crearíamos 20.000 carpetas con archivos diminutos, lo cual sería contraproducente (el llamado "problema de small files").

7. Schema Merging — 0.57 s

Para esta prueba dividimos el dataset en dos trozos con esquemas ligeramente distintos (al segundo le añadimos una columna extra) y los leímos juntos con `mergeSchema=true`. Spark tiene que abrir los metadatos de cada archivo Parquet, comparar los esquemas y construir uno unificado que incluya todas las columnas. Las filas del primer trozo, que no tenían la columna extra, aparecen con null en ella. Ese proceso de descubrimiento y unión de esquemas es lo que añade el tiempo extra (0.57 s). En producción esto es útil cuando un dataset va evolucionando y se le van añadiendo columnas nuevas con el tiempo.

8. Bucketing (JOIN) — 1.17 s

Es la segunda barra más alta del gráfico porque estamos haciendo un JOIN, que es la operación más cara en Big Data. Cruzamos la tabla consigo misma por `student_id`. Sin bucketing, Spark tendría que hacer un Shuffle completo: redistribuir todos los datos entre los nodos del clúster para que los registros con el mismo `student_id` acaben en la misma partición. Con `bucketBy(8, "student_id")`, los datos ya están pre-agrupados en 8 cubos por ID en disco, así que Spark se salta el shuffle y hace el join directamente. Aunque 1.17 s parece mucho comparado con las otras pruebas, en un dataset grande la diferencia entre hacer o no shuffle puede ser de minutos u horas.

Resultados finales

Formato	Tamaño	Tiempo	Ahorro espacio	Mejora velocidad
CSV (Original)	1.52 MB	5.20 s	—	—
Parquet (sin compresión)	527.30 KB	0.76 s	66.1%	85.4%
Parquet (Snappy)	527.47 KB	0.44 s	66.1%	91.5%
Parquet (Particionado)	617.67 KB	0.37 s	60.2%	92.8%